Step 4 - Wrap Up

In this chapter, we designed a scalable **chat system** that supports:

- One-to-one chat
- Small group chat
- Online presence
- Multiple device synchronization
- Push notifications
- Real-time messaging using WebSocket

The architecture contains several important components:

Component	Responsibility
Chat Servers	Real-time message delivery
Presence Servers	Manage online/offline status
Notification Servers	Send push notifications
API Servers	Login, signup, profile management
Key-Value Store	Store chat history
Service Discovery	Assign optimal chat servers
Message Queues	Reliable asynchronous processing

Key Design Decisions

1. WebSocket for Real-Time Communication

WebSocket is chosen because it provides:

- Persistent connection
- Bidirectional communication
- Low latency
- Efficient real-time updates

Unlike polling or long polling, WebSocket reduces:

- Network overhead
- Repeated HTTP requests
- Message delivery latency

2. Key-Value Store for Chat History

Key-value databases are preferred because they provide:

- Horizontal scalability
- Low latency reads/writes
- Efficient storage for massive message volume

Examples used in real systems:

- Facebook Messenger → HBase
- Discord → Cassandra

3. Multiple Device Synchronization

Each device maintains:

```
None
cur_max_message_id
```

This allows:

- Independent synchronization
- Fast recovery after reconnect
- Consistent chat history across devices

4. Push Notifications

If a user is offline:

- Push Notification servers notify the user
- Messages remain stored safely in KV storage

This ensures:

- Reliable delivery
- Better user engagement

5. Presence System

Presence servers track:

- Online status
- Last active time
- Heartbeats

Heartbeat mechanisms avoid unnecessary:

- Online/offline flickering
- Frequent reconnect updates

Additional Talking Points

If extra interview time is available, the following advanced topics can be discussed.

1. Media File Support

Current design only supports text messages.

To support:

- Images
- Videos
- Documents

additional optimizations are needed.

Important Concepts

Compression

Reduce media size before transmission.

Cloud Storage

Store large files in:

- AWS S3
- Google Cloud Storage
- CDN systems

Thumbnails

Generate smaller preview images/videos for faster loading.

2. End-to-End Encryption (E2EE)

Applications like WhatsApp support E2EE.

Benefits

Only:

- Sender
- Recipient

can read the messages.

Even servers cannot decrypt the content.

Challenges

E2EE introduces complexity in:

- Key management
- Multi-device synchronization
- Backup recovery

3. Client-Side Caching

Caching recent messages on the client side helps:

- Reduce server requests
- Improve loading speed
- Lower bandwidth usage

Frequently accessed chats can be loaded instantly.

4. Geographically Distributed Infrastructure

Applications like Slack improve performance using geographically distributed systems.

Benefits

- Lower latency
- Faster data retrieval
- Better user experience globally

Data such as:

- Channels
- User metadata
- Cached messages

can be replicated closer to users.

5. Error Handling

Robust error handling is critical in large-scale chat systems.

Chat Server Failure

A chat server may fail while maintaining:

```
None
hundreds of thousands of WebSocket connections
```

Recovery Mechanism

1. Service discovery detects failed server
2. Clients reconnect automatically
3. Zookeeper/service registry assigns a new chat server

This minimizes downtime.

6. Message Resend Mechanism

Messages may fail due to:

- Network failure
- Temporary server outage
- Packet loss

Common Solutions

Retry Logic

Failed messages are resent automatically.

Queueing

Messages remain in queues until delivery succeeds.

Acknowledgement System

Clients confirm successful receipt of messages.

Scalability Considerations

Large-scale chat systems require:

Technique	Purpose
Horizontal Scaling	Add more chat servers
Load Balancers	Distribute traffic

Replication	Improve availability
Sharding	Partition message data
CDN	Fast media delivery
Message Queues	Decouple services

Final Summary

The designed chat system achieves:

- Real-time communication
- Low latency
- Reliable delivery
- Online presence tracking
- Multiple device synchronization
- Scalability for millions of users

By combining:

- WebSocket
- Key-Value storage
- Service discovery
- Push notifications
- Presence servers

the system can efficiently support modern chat applications like:

- WhatsApp
- Messenger
- Slack
- Discord